

Sistemi Operativi – a.a. 2025/2026

prova di laboratorio

– 14 luglio 2026 –

Creare un programma `crc-records-checker.c` in linguaggio C che accetti invocazioni sulla riga di comando del tipo:

```
crc-records-checker <M-verifiers> <file-bin-1> ... <file-bin-N>
```

Il programma prende in input $N \geq 1$ file binari contenenti sequenze di record dotati di un checksum di integrità e dovrà individuare i record corrotti, verificando inoltre che il numero di record integri effettivamente rilevato coincida con quello dichiarato all'interno di ogni file.

Ogni file binario ha la seguente struttura:

- un header di 4 byte contenente un intero senza segno little-endian che dichiara il numero di record integri presenti nel file (usare il tipo `C uint32_t` in `stdint.h`);
- una sequenza di record binari di 16 byte ciascuno: i primi 15 byte rappresentano i dati, l'ultimo byte rappresenta il checksum del record; i byte sono da intendersi senza segno e quindi da trattare con il tipo `C uint8_t` (o `unsigned char`).

Un record si dice **integro** se il byte di checksum coincide con la somma dei 15 byte di dati modulo 256; in caso contrario il record si dice **corrotto**. Il numero di record contenuti in un file si ricava dalla dimensione dello stesso: $(\text{dimensione} - 4) / 16$.

Sono disponibili dei [file di esempio](#): `records-A.bin`, `records-B-bad.bin`, `records-C.bin`.

In particolare al suo avvio il programma creerà $N + M$ thread ausiliari:

- N thread **lettori** che si occuperanno, rispettivamente, di leggere uno dei file indicati sulla riga di comando; il file dovrà essere letto tramite la mappatura dei file in memoria (`mmap`): qualunque altro metodo sarà considerato errato; ogni lettore depositerà preliminarmente nel vettore condiviso il numero di record integri dichiarato nell'header e, per ogni record letto, inserirà un elemento nella coda candidati;
- M thread **verificatori** che si occuperanno, in modo concorrente, di estrarre, uno alla volta, i record dalla coda candidati e di ricalcolarne il checksum: se questo coincide con quello dichiarato nel record, sarà incrementato il contatore degli integri del file di provenienza; in caso contrario sarà incrementato il contatore dei corrotti e il record sarà inserito nella coda scarti destinata al thread principale.

Le strutture dati condivise saranno:

- una coda **candidati** di elementi con capienza massima di 10 elementi che conterrà i record letti dai file da verificare;
- una coda **scarti** di elementi con capienza massima di 5 elementi che conterrà solo i record risultati corrotti;
- un vettore di N terne `<dichiarati, integri, corrotti>` con i conteggi relativi a ciascun file in input;
- mutex e variabili condizione: il loro numero, comunque minimale, e le modalità d'uso sono da determinare da parte dello studente;
- eventuali flag/contatori di fine-lavoro.

Il singolo elemento delle code conterrà le seguenti informazioni:

- i 16 byte del record (15 di dati + 1 di checksum);
- l'identificativo del file di provenienza;
- il numero progressivo del record all'interno del file;
- il checksum ricalcolato (significativo solo per la coda scarti).

Il thread principale/padre si occuperà di estrarre e visualizzare i record corrotti inseriti nella coda scarti; al termine, per ogni file, confronterà il numero di record integri rilevato con quello dichiarato nell'header riportando l'esito della verifica automatica (superata o fallita) e il riepilogo

finale.

Tutti i thread dovranno terminare spontaneamente alla fine dei lavori e non si dovranno usare strutture dati con visibilità globale. È necessario rispettare fedelmente la struttura dell'output riportato nell'esempio a seguire. Codici sorgente con errori che bloccano la compilazione e pregiudicano la generazione del codice binario con compilatori standard (gcc o clang) non saranno valutati.

Tempo: 2 ore e 30 minuti

La struttura dell'output atteso è la seguente:

```
$ ./crc-records-checker 2 records-A.bin records-B-bad.bin records-C.bin
[MAIN] creazione di 3 thread lettori e 2 thread verificatori
[READER-1] file 'records-A.bin': 100 record totali, 88 dichiarati integri
[READER-1] record candidato n.1 (offset 4)
[READER-1] record candidato n.2 (offset 20)
[READER-1] record candidato n.3 (offset 36)
[READER-2] file 'records-B-bad.bin': 80 record totali, 71 dichiarati integri
[READER-2] record candidato n.1 (offset 4)
[READER-3] file 'records-C.bin': 120 record totali, 103 dichiarati integri
[READER-3] record candidato n.1 (offset 4)
[VERIF-1] record n.1 del file 'records-A.bin': checksum dichiarato 66, calcolato 66 -> integro
[VERIF-1] record n.2 del file 'records-A.bin': checksum dichiarato 146, calcolato 27 -> corrotto
[VERIF-1] record n.3 del file 'records-A.bin': checksum dichiarato 253, calcolato 244 -> corrotto
[MAIN] record corrotto: n.2 del file 'records-A.bin' (checksum dichiarato 146, calcolato 27)
[VERIF-1] record n.1 del file 'records-B-bad.bin': checksum dichiarato 96, calcolato 96 -> integro
[READER-3] record candidato n.2 (offset 20)
[VERIF-2] record n.1 del file 'records-C.bin': checksum dichiarato 28, calcolato 28 -> integro
...
[READER-2] terminazione con 80 record letti
[READER-3] terminazione con 120 record letti
[READER-1] terminazione con 100 record letti
[VERIF-1] terminazione con 160 record verificati
[VERIF-2] terminazione con 140 record verificati
[MAIN] file 'records-A.bin': 88 integri e 12 corrotti su 100 record -> verifica automatica superata
(dichiarati 88)
[MAIN] file 'records-B-bad.bin': 70 integri e 10 corrotti su 80 record -> verifica automatica fallita
(dichiarati 71, rilevati 70)
[MAIN] file 'records-C.bin': 103 integri e 17 corrotti su 120 record -> verifica automatica superata
(dichiarati 103)
[MAIN] verifiche superate: 2/3
[MAIN] terminazione
```